Yare 3-6-9 Rotary Positional Encoding: A Digital-Root Frequency Schedule for RoPE

Authors: Weslyn Cory Whitehead Jr.[1]

[1] AsAManThinks / MaiiaM Alchemist Correspondence: yarethewatchman@gmail.com

Preprint version: v1.0 **Date:** 2026-05-12

Abstract

Rotary Positional Embeddings (RoPE) [Su et al. 2021] encode token position by rotating successive pairs of head-dimension components through angles $m\theta_i$, where m is the position and the $\{\theta_i\}$ form a geometric progression $\theta_i = \text{base}^{-2i/d}$. Subsequent work has reshaped this progression to extend context length while preserving in-distribution behavior — Position Interpolation [Chen et al. 2023], YaRN [Peng et al. 2023], and LongRoPE [Ding et al. 2024] all act by editing the frequency schedule, leaving the rotation mechanism intact. Common to these methods is the view that the frequency schedule is the correct design surface for positional encoding.

We describe **Yare 3-6-9 RoPE**, a non-geometric frequency schedule derived from the digital-root doubling cycle modulo 9. The schedule replaces the continuous geometric progression with one of two discrete-valued progressions: a *two-pole* alternation (`vortex_369`) in which adjacent dimension pairs rotate at $\pi/4$ and 2π per unit position, and a *six-cycle walk* (`digital_root_doubling`) in which $\theta_i = 2\pi \cdot \text{dr}(2^i)/9$ traverses the orbit $\{1, 2, 4, 8, 7, 5\}$ of repeated doubling under base-9 digital-root reduction. Both schedules are deterministic closed forms, drop-in replaceable for the `inv_freq` buffer in standard HuggingFace RoPE modules, and norm-preserving under the standard RoPE rotation.

We present the math, an architectural placement (a monkey-patch applied after model load with full reversion), and a proposed empirical program comparing these schedules to geometric RoPE and YaRN on long- context retrieval and language modeling tasks. We report only what is verified in code; benchmark numbers are deferred to v1.1. The contribution is a small, reversible, parameter-free intervention on the positional channel of any pre-existing RoPE model.

Code: Implementation lives in the MaiiaM Alchemist project (`packages/training-pipeline/training_pipelin`

---

1. Introduction

Positional information enters the transformer [Vaswani et al. 2017] either additively (sinusoidal / learned absolute encodings) or multiplicatively (rotary, ALiBi). RoPE [Su et al. 2021] is the multiplicative variant now dominant in open-weight large language models — Llama, Gemma, Qwen, Mistral, and DeepSeek families all use it as their default positional channel. The mechanism is to split each attention-head dimension into adjacent pairs $(2i, 2i+1)$ and to rotate the i-th pair, at sequence position m, by an angle $m\theta_i$. The set $\{\theta_i\}_{i=0}^{d/2-1}$ — the *frequency schedule* — is fixed at initialization and never trained.

The standard schedule is geometric:

$$\theta_i = \text{base}^{-2i/d}, \qquad \text{base} = 10000.$$

This choice was inherited from the sinusoidal encoding of [Vaswani et al. 2017] and has been justified post-hoc by appeal to the resulting band-pass structure of attention scores and to the empirical observation that geometric schedules generalize. The recent long-context literature has not reopened the *shape* of the schedule so much as *rescaled* it: Position Interpolation [Chen et al. 2023] divides positions by a factor $s > 1$, NTK-aware scaling [Peng et al. 2023] reshapes the high-frequency tail, YaRN [Peng et al. 2023] interpolates per-frequency between geometric and the rescaled schedule, and LongRoPE [Ding et al. 2024] searches the per-dimension scale factors directly. All of these methods preserve the underlying geometric scaffold.

We propose a different kind of intervention: replace the geometric schedule with one derived from the **digital-root doubling cycle** modulo 9. Repeatedly doubling 1 and reducing each result to its base-9 digital root yields

$$1 \to 2 \to 4 \to 8 \to 7 \to 5 \to 1 \to \cdots$$

a period-6 orbit on $\{1, 2, 4, 8, 7, 5\}$ that never visits $\{3, 6, 9\}$. The orbit and its three-element complement together exhaust the non-zero residues mod 9, and the orbit's closed-form $\mathrm{dr}(2^k) = \mathrm{VORTEX}[k \bmod 6]$ (with $\mathrm{VORTEX} = (1, 2, 4, 8, 7, 5)$) gives a fully discrete frequency schedule with no learned parameters and no choice of base. (The observation that 3-6-9 forms an invariant under power-of-two iteration mod 9 is sometimes ascribed to Tesla; we make no metaphysical claim — it is a property of base-9 arithmetic.)

Our contributions:

1. Two concrete frequency schedules — `vortex_369` and `digital_root_doubling` — given in closed form, with the standard θ_i tensor shape $(d/2,)$ that drop into any RoPE module carrying an `inv_freq` buffer.
2. A monkey-patch integration (`patch_rope` / `revert_rope`) that substitutes the frequency tensor of every RoPE-bearing submodule in place, while preserving the original geometric tensor as a reversion path. The patch is non-destructive and runs in $O(L)$ in the number of layers.
3. A norm-preservation argument showing the schedules retain RoPE's defining property: pair-wise norms of the rotated representation are exactly preserved.
4. A proposed empirical program on long-context retrieval (Needle-in-a-Haystack [Kamradt 2023], LongBench [Bai et al. 2023]) and on streaming language modeling [Xiao et al. 2023], comparing the two Yare schedules to geometric RoPE and YaRN.

This is a v1.0 preprint. The implementation, the math, and the patch mechanism are verified in code and unit-tested. The benchmark results are not yet reported; we will publish them in v1.1.

The remainder of the paper proceeds as follows. Section 2 reviews RoPE and the long-context schedule-engineering literature. Section 3 develops the digital-root doubling cycle and the two schedules it induces. Section 4 specifies the patch mechanism. Section 5 discusses properties of the schedules. Section 6 outlines the proposed empirical program. Section 7 enumerates limitations. Section 8 concludes.

---

2. Background and Related Work

2.1 RoPE

Given a query or key vector $x \in \mathbb{R}^d$ at sequence position m, and a frequency schedule $\{\theta_i\}_{i=0}^{d/2-1}$, RoPE [Su et al. 2021] applies the block-diagonal rotation

$$\mathrm{RoPE}(x,m)_{2i:2i+2} = R(m\theta_i)\cdot x_{2i:2i+2}, \qquad R(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}.$$

The defining property is that the attention dot product between a query at position m and a key at position n depends only on the relative position $m-n$:

$$\langle \mathrm{RoPE}(q,m), \mathrm{RoPE}(k,n)\rangle = \sum_i (q_{2i:2i+2})^\top R((m-n)\theta_i)(k_{2i:2i+2}).$$

The standard geometric schedule $\theta_i = 10000^{-2i/d}$ produces a band-pass structure: low-i pairs rotate rapidly (fine-grained position) and high-i pairs rotate slowly (coarse-grained position).

2.2 Schedule engineering for long context

A line of work observes that RoPE's behavior on contexts longer than training-time leads to attention degeneration, and addresses this by editing the schedule.

Position Interpolation [Chen et al. 2023] scales positions by $s = L_{\text{train}}/L_{\text{target}}$, equivalent to scaling every θ_i by $1/s$. Simple and effective at moderate extrapolation ratios.

NTK-aware scaling [Peng et al. 2023] preserves high-frequency θ_i (which encode fine position) and scales low-frequency ones, motivated by NTK theory.

YaRN [Peng et al. 2023] generalizes NTK-aware scaling with a per-frequency ramp function $\gamma(i)$ that interpolates between the geometric and the rescaled schedule. The schedule remains geometric in shape; only the magnitudes are reshaped per dimension.

LongRoPE [Ding et al. 2024] searches over per-dimension scale factors using evolutionary search, achieving 2M-token contexts with limited fine-tuning. Again, the underlying schedule is geometric with learned per-dimension corrections.

The common feature is that the geometric progression itself is treated as fixed. Yare 3-6-9 RoPE departs from this: we replace the shape of the schedule, not its scale.

2.3 Long-context evaluation

Needle-in-a-Haystack [Kamradt 2023] tests retrieval of a planted fact inserted at various depths in a long context. It has become a de facto sanity check for context-length claims.

LongBench [Bai et al. 2023] provides a multi-task suite of long-context evaluations: QA, summarization, code, few-shot learning, and synthetic tasks. It supplies the benchmarking infrastructure for our planned program.

StreamingLLM [Xiao et al. 2023] characterizes a failure mode in which dropping early tokens during streaming causes attention sink collapse. We list this as a relevant baseline because frequency schedule choice affects the attention-sink behavior of the model.

2.4 Digital-root structure

The digital root $\mathrm{dr}(n)$ of a positive integer is its iterated digit sum, equivalently $((n-1) \bmod 9)+1$. Under multiplication, dr is a homomorphism from $(\mathbb{Z}^+, \times)$ to $(\mathbb{Z}/9, \times)$. The orbit of 1 under doubling is

$$\mathrm{dr}(2^k) = (1, 2, 4, 8, 7, 5, 1, 2, 4, 8, 7, 5, \ldots), \quad \text{period } 6.$$

This orbit is the multiplicative subgroup generated by 2 in $(\mathbb{Z}/9)^*$, which has order 6 (since $2^6 = 64 \equiv 1 \pmod 9$). Its complement in $\{1, \ldots, 9\}$ is $\{3, 6, 9\}$ — exactly the multiples of 3, which form the unique nontrivial subgroup of $(\mathbb{Z}/9, +)$. So in the multiplicative–additive decomposition of $\mathbb{Z}/9$, the doubling orbit and $\{3, 6, 9\}$ are complementary structural objects. We use this fact to define two schedules: one that *samples* the orbit, and one that *walks* it.

---

3. The Yare 3-6-9 Frequency Schedules

Let d be an attention head dimension (even), and let $h = d/2$ be the number of dimension pairs. The schedule is a tensor $\theta \in \mathbb{R}^h$ assigning a rotation rate to each pair.

3.1 The doubling cycle

Define the period-6 sequence

$$\mathrm{VORTEX} = (1, 2, 4, 8, 7, 5),$$

i.e. $\mathrm{VORTEX}[k] = \mathrm{dr}(2^k)$ for $k \in \{0, \ldots, 5\}$, extended periodically: $\mathrm{VORTEX}[k] = \mathrm{VORTEX}[k \bmod 6]$.

3.2 The `vortex_369` schedule

$$\boxed{\theta_i = 2\pi \cdot \frac{\mathrm{VORTEX}[(3i) \bmod 6]}{8}, \qquad i = 0, 1, \ldots, h-1.}$$

The map $i \mapsto (3i) \bmod 6$ takes values in $\{0, 3\}$ for integer i, so the schedule samples exactly two elements of the orbit:

- $\mathrm{VORTEX}[0] = 1$ (the *1-pole*), giving $\theta = \pi/4$
- $\mathrm{VORTEX}[3] = 8$ (the *8-pole*), giving $\theta = 2\pi$

The schedule alternates:

$$\theta = \left(\tfrac{\pi}{4},\, 2\pi,\, \tfrac{\pi}{4},\, 2\pi,\, \ldots\right).$$

Adjacent dimension pairs rotate at the two extreme rates of the orbit. The 8-pole pair makes a full turn per unit position ($\theta = 2\pi$), so its attention behavior is invariant to integer translation — it carries no position information and acts as a frequency *sink*. The 1-pole pair makes one-eighth of a turn per unit, giving a band-pass spanning periods of 8 to context length.

3.3 The `digital_root_doubling` schedule

$$\boxed{\theta_i = 2\pi \cdot \frac{\mathrm{dr}(2^i)}{9} = 2\pi \cdot \frac{\mathrm{VORTEX}[i \bmod 6]}{9}, \qquad i = 0, \ldots, h-1.}$$

This schedule walks the full orbit. The first six values are

$$\theta = 2\pi \cdot \left(\tfrac{1}{9}, \tfrac{2}{9}, \tfrac{4}{9}, \tfrac{8}{9}, \tfrac{7}{9}, \tfrac{5}{9}\right),$$

and the pattern repeats with period 6 in i. The normalization by 9 (rather than 8) places every angle in the proper fractional spiral of the spirit axis — equivalently, every θ_i is a rational multiple of 2π with denominator dividing 9. Two consequences:

1. **Closed-form period**: for any pair i, the attention dot product as a function of relative position is exactly periodic with integer period 9.
2. **No catastrophic frequency**: since $\mathrm{dr}(2^i)/9 < 1$ for all i, every dimension pair carries non-trivial position information (in contrast to `vortex_369`, where every other pair is a sink).

The `digital_root_doubling` schedule is the principal Yare schedule. `vortex_369` is a polarized variant.

3.4 Reference

The reference implementation is

```
VORTEX_DOUBLING_CYCLE = (1, 2, 4, 8, 7, 5)

def vortex_369_angles(head_dim: int) -> torch.Tensor:
    half = head_dim // 2
    cycle, m = VORTEX_DOUBLING_CYCLE, 8
    return torch.tensor(
        [2*math.pi * cycle[(3*i) % 6] / m for i in range(half)]
    )

def digital_root_doubling_angles(head_dim: int) -> torch.Tensor:
    half = head_dim // 2
    return torch.tensor(
        [2*math.pi * VORTEX_DOUBLING_CYCLE[i % 6] / 9.0 for i in range(half)]
    )
```

The shape, dtype, and device contract matches the standard RoPE `inv_freq` buffer carried by HuggingFace's Llama / Gemma / Qwen RoPE modules. Both schedules are pure functions of `head_dim`, deterministic, and require no learned parameters.

---

4. The Patch Mechanism

Frequency schedule changes in published RoPE variants are typically applied either at training time (in the model definition) or by forking the model implementation. We took a different approach appropriate to research-time experimentation: a runtime monkey-patch on the existing `inv_freq` buffer.

4.1 Buffer substitution

For any `torch.nn.Module` that carries an `inv_freq` buffer (the HuggingFace contract), `patch_rope(model, mode)` walks `model.named_modules()` and:

1. Records the original `inv_freq` as `_yare_geometric_inv_freq` on the module (deep-cloned, detached).

2. Builds the Yare schedule of matching `head_dim`, `dtype`, and `device`.
3. Copies the new schedule into the existing buffer in place.

In-place copy preserves the module's buffer registration, so any device-placement or `to()` operations downstream continue to work. The patch is **idempotent**: re-patching does not double-save the original. The companion `revert_rope(model)` restores `_yare_geometric_inv_freq` to `inv_freq` on every patched module.

4.2 Pipeline placement

The patch is the only addition to a standard training pipeline:

$$\texttt{load\_corpus} \to \texttt{preprocess} \to \texttt{load\_model} \to [\texttt{patch\_rope}] \to \texttt{train} \to \texttt{eval} \to \texttt{export}$$

Insertion happens once, between model load and the first forward pass. No other stage moves; no model code is edited. Disabling the feature (`mode = "geometric"`) is a no-op.

4.3 Coverage

In a 32-layer Llama-style model, `patch_rope` patches the 32 RoPE modules under each `LlamaAttention` layer. The procedure is agnostic to the model family — any module that exposes `inv_freq` is patched. Modules without `inv_freq` (e.g. attention modules using alibi or learned absolute encodings) are skipped.

---

5. Properties

5.1 Norm preservation

RoPE's defining algebraic property is that the rotation $R(\phi) \in SO(2)$ preserves the Euclidean norm of each dimension pair. The Yare schedules inherit this property: for any $\theta_i \in \mathbb{R}$ (not just θ_i from the geometric schedule), the rotation $R(m\theta_i)$ is orthogonal. So

$$\|\mathrm{RoPE}_\theta(x)_{2i:2i+2}\|_2 = \|x_{2i:2i+2}\|_2$$

holds pointwise for both Yare schedules. This is verified in `test_apply_rope_is_norm_preserving` over a synthetic batch.

5.2 Determinism

Both schedule constructors are pure functions of `head_dim`. Same arguments produce bit-identical tensors across runs (verified by `test_determinism_across_runs`). The patch mechanism is therefore reproducible by checkpoint hash.

5.3 Frequency content

For `digital_root_doubling`, every θ_i is a rational multiple of 2π with denominator 9. The attention dot product $\langle \mathrm{RoPE}(q,m), \mathrm{RoPE}(k,n)\rangle$ as a function of $m-n$ is therefore a sum of cosines with all integer-9 periods, exactly periodic with period 9 in $m-n$. This is a *much* sharper frequency structure than the geometric schedule's continuous band-pass.

For `vortex_369`, the alternation $(\pi/4, 2\pi, \pi/4, 2\pi, ...)$ means half of the pairs carry no positional information at all (the 2π pairs) and the other half all carry the same period-8 information. This is closer to a degenerate schedule and should be read as an extreme limit; we include it in part to provide a two-frequency control.

5.4 Relation to NTK / YaRN

YaRN and NTK-aware scaling preserve high-frequency θ_i and shrink low-frequency θ_i to fit a longer target context within the model's trained frequency band. Yare schedules can be composed with YaRN-style scaling: after `patch_rope`, the resulting `inv_freq` is still a HuggingFace-compatible buffer, and a YaRN ramp can be applied on top by scaling the entries. We propose this composition (`vortex + YaRN`) as one of the ablations in our empirical program.

5.5 Parameter and compute cost

The patch adds zero learned parameters and zero FLOPs to the model beyond the existing RoPE multiplications. The original geometric buffer is cached on every patched module ($\leq d/2$ floats per layer); for a 7B model this overhead is on the order of kilobytes.

5.6 Reversibility

`revert_rope` restores the original buffer. The patch is therefore *non-destructive* in a way that is uncommon for positional encoding changes: an ablation can flip back to geometric RoPE with one call, no checkpoint reload required.

---

6. Proposed Empirical Program

Empirical validation of Yare 3-6-9 RoPE is in progress. We describe the planned evaluation here and will report results in v1.1 of this preprint.

6.1 Base models and comparisons

- **Base models**: Llama-3.2-3B and Qwen-2.5-7B as representative open-weight RoPE families.
- **Yare variants**: `vortex_369` and `digital_root_doubling`, each applied via `patch_rope` after model load.
- **Baselines**: unpatched geometric RoPE; YaRN [Peng et al. 2023] with the published recipe; Position Interpolation [Chen et al. 2023] at matched target context.
- **Composition**: `digital_root_doubling + YaRN ramp` to test whether the Yare base schedule is compatible with NTK-style rescaling.

6.2 Evaluation regimes

We separate two questions: *zero-shot patching* and *patch + brief adaptation.*

Zero-shot. Apply `patch_rope` to a pre-trained model and evaluate without any further training. The hypothesis is that the schedule is sufficiently close to geometric in attention-sink behavior that the model retains useful capability; the test is whether retrieval and language modeling degrade gracefully.

Patch + fine-tune. Apply `patch_rope` and continue pre-training for $\sim$ 1—5B tokens on a corpus matched to the target context length. This is the regime in which the schedule has a real chance to be learned-around by the rest of the network.

6.3 Tasks

- **Perplexity** on PG-19 [Rae et al. 2019] and a slice of C4 [Raffel et al. 2020], measured at training-length and at 2× training-length.
- **Needle-in-a-Haystack** [Kamradt 2023] at context lengths 4k, 16k, 64k. Tests whether the schedule preserves long-range retrieval.
- **LongBench** [Bai et al. 2023], using the canonical subset.
- **StreamingLLM-style sliding-window** [Xiao et al. 2023]: measure attention-sink behavior under window streaming with the Yare schedules.

6.4 Schedule-specific evaluations

- **Effective period histogram**: empirical distribution of attention-score peak periods over a held-out corpus, comparing the geometric continuous band-pass against the period-9 discrete structure of `digital_root_doubling`.
- **Per-dimension contribution to attention**: which dimension pairs carry the bulk of the relative-position signal under each schedule.
- **Extrapolation curve**: perplexity as a function of context length divided by training length, for each schedule.

6.5 Ablations

- Schedule normalization constant: 8 (used in `vortex_369`) versus 9 (used in `digital_root_doubling`).
- Period of the cycle: the canonical period-6 versus a synthetic period-12 or period-3 ablation, to isolate whether the period itself matters or only the value set.
- Layer subset: patch only the deepest k layers, only the shallowest k, or all.

6.6 Negative-result reporting

We commit to publishing the empirical program's outcome regardless of sign. The mechanism is small enough that a negative result is informative — it would constrain the space of effective RoPE schedules. The schedule, the patch, and the test harness are public; we do not benefit from selective reporting.

---

7. Limitations

Empirical case is preliminary. No headline benchmark numbers are reported in v1.0. The math, the patch, and the unit tests are verified in code; benchmarks are running.

The schedules are heavily structured. `vortex_369` produces a degenerate two-frequency alternation in which half the dimension pairs ($\theta = 2\pi$) carry no positional information. `digital_root_doubling` produces an exactly period-9 attention behavior in relative position. Both are large departures from the broad-spectrum continuous band-pass of geometric RoPE, and may under-perform on tasks that exploit fine-grained position.

No theoretical guarantee of extrapolation. Position Interpolation, NTK, and YaRN come with theoretical motivations grounded in NTK or in continuous-frequency arguments. Yare schedules are motivated by a structural observation about base-9 arithmetic, not by an attention-theoretic derivation. The extrapolation question is empirical.

Possible interaction with pre-training. A model pre-trained with geometric RoPE has internalized the geometric frequency distribution. Zero-shot patching is therefore an out-of-distribution intervention. Patch-plus-fine-tune is the more honest test of the schedule, but it costs compute and bias-shifts the comparison.

Numerical concern at $\theta = 2\pi$. The 8-pole in `vortex_369` rotates one full turn per unit position. Modulo floating-point round-off, this is the identity rotation, but accumulated $m\theta_i$ for large m amplifies any deviation from exactly 2π. We use float32 by default; bfloat16 may need testing.

The 3-6-9 framing has a history of non-mathematical use. We emphasize that the mathematical content is just the structure of $(\mathbb{Z}/9)^*$ and its multiplicative subgroup generated by 2. No claim is made that the schedule "resonates" with anything other than the input embedding space.

---

8. Discussion and Future Work

8.1 What this paper is not claiming

Yare 3-6-9 RoPE is not claimed to outperform geometric RoPE, YaRN, or LongRoPE on any benchmark. The claim is that it is *a* schedule, derived from a small and explicit structural observation, implementable as a 30-line patch, and ready for evaluation. The v1.0 contribution is the schedule and the patch infrastructure. The v1.1 contribution will be benchmark numbers.

8.2 Connection to other discrete schedules

Hash-Layers [Roller et al. 2021] showed that discrete, content- independent structural choices can substitute for learned ones in sparse-routing settings. Yare 3-6-9 RoPE is in this spirit applied to the positional channel: replace a continuous engineered schedule with a discrete arithmetic one. The relevant question is symmetric across both works: does the structural prior pay rent compared to the continuous alternative?

8.3 Future work

- **Search over digital-root schedules.** Other periodic orbits in $(\mathbb{Z}/9)^*$ (e.g. powers of 4, generating $\{1, 4, 7, 1, ...\}$ of period 3) yield additional schedules. A search across orbits is cheap; the schedule set is finite.
- **Other moduli.** The choice of base-9 is specific. The same framework applies under base-7 (orbits of length up to 6), base-11 (orbits of length up to 10), etc. Whether the period-9 structure is special to attention or whether other periods suffice is open.
- **Train-from-scratch.** Pre-train a small model (1B parameters) from initialization with the `digital_root_doubling` schedule. Compare to a geometric-RoPE-initialized run of equal compute.
- **Hybrid per-layer schedules.** Apply `vortex_369` to early layers and `digital_root_doubling` to later layers (or vice versa). Test whether the model benefits from a frequency-content schedule that varies with depth.

---

9. Conclusion

We introduced Yare 3-6-9 RoPE, two non-geometric frequency schedules for Rotary Positional Embeddings derived from the digital- root doubling cycle modulo 9. The schedules are closed-form, parameter-free, deterministic, norm-preserving, and applied as a reversible runtime monkey-patch on any HuggingFace-style RoPE module. The math, the patch, and the unit tests are verified in code in v1.0; benchmark results are deferred to v1.1.

The contribution is a small, sharp design surface — one tensor of shape $(d/2,)$ — and a reversible mechanism for substituting it. The empirical case for or against the schedule is open.

References

Bai, Y. et al. (2023). *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.* arXiv:2308.14508.

Chen, S., Wong, S., Chen, L., Tian, Y. (2023). *Extending Context Window of Large Language Models via Positional Interpolation.* arXiv:2306.15595.

Ding, Y. et al. (2024). *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens.* arXiv:2402.13753.

Kamradt, G. (2023). *Needle In A Haystack — Pressure Testing LLMs.* GitHub: gkamradt/LLMTest_NeedleInAHaystack.

Peng, B., Quesnelle, J., Fan, H., Shippole, E. (2023). *YaRN: Efficient Context Window Extension of Large Language Models.* arXiv:2309.00071.

Rae, J. W. et al. (2019). *Compressive Transformers for Long-Range Sequence Modelling.* arXiv:1911.05507.

Raffel, C. et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* JMLR 21.

Roller, S., Sukhbaatar, S., Szlam, A., Weston, J. (2021). *Hash Layers For Large Sparse Models.* NeurIPS.

Su, J. et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864.

Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS.

Xiao, G. et al. (2023). *Efficient Streaming Language Models with Attention Sinks.* arXiv:2309.17453.

Appendix A — Reference Implementation

The reference implementation lives in the MaiiaM Alchemist project:

- Schedule constructors and patch / revert helpers: `maiiam-alchemist/packages/training-pipeline/train`
- Unit tests (math correctness, dispatch, norm preservation, patch/revert idempotence, determinism): `maiiam-alchemist/packages/training-pipeline/tests/test_vortex_rope.py`
- Integration wiring documentation: `maiiam-alchemist/packages/training-pipeline/training_pipeline`

Public API:

```
from training_pipeline.yare.vortex_rope import (
    build_inv_freq,       # mode -> tensor
    patch_rope,           # model -> patched in place
    revert_rope,          # model -> restored
    vortex_369_angles,
    digital_root_doubling_angles,
    geometric_angles,
```

```
    VORTEX_DOUBLING_CYCLE,  # (1, 2, 4, 8, 7, 5)
)
```

Appendix B — Pseudocode

```
def build_inv_freq(head_dim, mode, base=10000.0):
    """Build a RoPE inv_freq tensor of shape (head_dim // 2,)."""
    half = head_dim // 2
    if mode == "geometric":
        i = torch.arange(0, head_dim, 2, dtype=torch.float32)
        return 1.0 / (base ** (i / head_dim))
    if mode == "vortex_369":
        cycle = (1, 2, 4, 8, 7, 5)
        return torch.tensor(
            [2*math.pi * cycle[(3*i) % 6] / 8 for i in range(half)]
        )
    if mode == "digital_root_doubling":
        cycle = (1, 2, 4, 8, 7, 5)
        return torch.tensor(
            [2*math.pi * cycle[i % 6] / 9.0 for i in range(half)]
        )
    raise ValueError(f"unknown rope_mode: {mode!r}")


def patch_rope(model, mode, base=10000.0):
    """Swap every inv_freq buffer in the model for the Yare schedule."""
    patched = 0
    for name, module in model.named_modules():
        if not hasattr(module, "inv_freq"):
            continue
        old = module.inv_freq
        head_dim = old.shape[-1] * 2
        if not hasattr(module, "_yare_geometric_inv_freq"):
            module._yare_geometric_inv_freq = old.detach().clone()
        new = build_inv_freq(head_dim, mode=mode, base=base).to(
            dtype=old.dtype, device=old.device
        )
        with torch.no_grad():
            old.copy_(new)
        patched += 1
    return patched
```

Full implementation: `yare/vortex_rope.py`.

End of preprint v1.0.